



پاسخ‌ها لزوماً یکتا نیستند و پاسخ‌های درست دیگر نیز در نظر گرفته می‌شوند.

(۱)

برای حل این سوال، در ابتدا، مروری بر انواع ایجنت‌ها خواهیم داشت و سپس با توجه به توضیحات ارائه شده، به جواب سوالات می‌پردازیم. عامل‌ها بر اساس میزان هوشمندی، پیچیدگی فرآیند تصمیم‌گیری و نحوه تعامل با محیط، به چهار دسته اصلی تقسیم می‌شوند:

۱. عامل واکنشی ساده (Simple Reflex Agent)

این نوع عامل، ساده‌ترین ساختار را دارد و تصمیمات خود را صرفاً بر اساس Current Percept اتخاذ می‌کند. این عامل فاقد حافظه است و تاریخچه‌ی ادراکات قبلی را در نظر نمی‌گیرد. عملکرد آن بر پایه‌ی مجموعه‌ای از قواعد Condition-Action Rules است.

- مثال: ترموستات نمونه‌ی کلاسیکی است؛ اگر دما از حد مجاز کمتر شود، سیستم گرمایشی فعال می‌شود، بدون آن‌که حالت‌های قبلی محیط را لحاظ کند.
- ویژگی محیط: این عامل در محیط‌های Fully Observable عملکرد مطلوب دارد؛ نوع محیط (پویا یا ایستا) بر کارایی آن اثری ندارد و تنها کافی است محیط به طور کامل قابل مشاهده باشد.

۲. عامل واکنشی مدل‌گرا (Model-based Reflex Agent)

این عامل برای غلبه بر محدودیت محیط‌های جزئاً مشاهده‌پذیر، از یک Internal State یا حافظه بهره می‌برد. این حالت داخلی امکان مدل‌سازی بخش‌های دیده‌نشده‌ی محیط و ثبت تاریخچه‌ای از تغییرات را فراهم می‌کند.

- مثال: تشخیص چشمک‌زن بودن خودرو نیازمند دانستن وضعیت قبلی چراغ‌هاست؛ بنابراین عامل باید حالت «خاموش بودن» را به خاطر بسپارد تا بتواند «چشمک زدن» را درک کند.
- ویژگی محیط: این عامل در محیط‌های Deterministic کارایی دارد. در صورت وجود عدم قطعیت، مدل داخلی (که معمولاً به صورت گراف وضعیت در نظر گرفته می‌شود) در پیش‌بینی رفتار محیط ناکارآمد خواهد بود.

۳. عامل هدف‌گرا (Goal-based Agent)

عامل هدف‌گرا علاوه بر داشتن مدل داخلی از محیط، شامل تعریف دقیق یک یا چند Goal است. این عامل از میان مجموعه‌ی اعمال ممکن، آن‌هایی را انتخاب می‌کند که وضعیت فعلی را به وضعیت هدف برسانند. چنین تصمیم‌سازی‌هایی معمولاً با روش‌های Search و Planning انجام می‌شود.

۴. عامل مبتنی بر سودمندی (Utility-based Agent)

عامل هدف‌گرا تنها بین رسیدن یا نرسیدن به هدف تمایز قائل می‌شود، اما در مسائل واقعی، کیفیت و میزان مطلوبیت رسیدن به هدف نیز اهمیت دارد. عامل مبتنی بر سودمندی از یک Utility Function برای ارزیابی و مقایسه‌ی وضعیت‌های مختلف استفاده می‌کند.

- ضرورت: در مسائل دارای اهداف متعدد یا متناقض، این عامل امکان برقراری تعادل بهینه (Trade-off) میان اهداف را فراهم می‌کند.

- **مثال:** در سیستم تاکسی خودران، عامل باید هم‌زمان بین اهدافی چون کاهش مصرف سوخت، رعایت ایمنی، کاهش زمان سفر و راحتی مسافر تعادل برقرار کند. از آن‌جا که عامل هدف‌گرای ساده نمی‌تواند چنین توازنی را برقرار کند، این سیستم از نوع عامل مبتنی بر سودمندی است.

جدول خلاصه‌ی مقایسه‌ی انواع عامل‌ها:

نوع عامل	مبنای تصمیم‌گیری	وضعیت حافظه	محیط مناسب
واکنشی ساده	ادراک لحظه‌ای	بدون حافظه	Fully Observable
واکنشی مدل‌گرا	ادراک + حالت داخلی	دارای حافظه	جزئاً مشاهده‌پذیر و Deterministic
هدف‌گرا	وضعیت + هدف	دارای حافظه	محیط‌های نیازمند Planning
مبتنی بر سودمندی	وضعیت + بیشینه‌سازی مطلوبیت	دارای حافظه	محیط‌های پیچیده با اهداف متناقض

اکنون به سوالات پاسخ می‌دهیم:

- الف) با توجه به توضیح ارائه شده، درست است.
- ب) با توجه به توضیح ارائه شده، عملکرد عامل واکنشی ساده مستقل از پویا یا ایستا بودن محیط است و بنابراین این مورد نادرست است.
- پ) درست است. به توضیحات ارائه شده رجوع شود.
- ت) نادرست است؛

- ایستا (Static): در طول تصمیم‌گیری عامل، وضعیت محیط تغییر نمی‌کند و محیط منتظر اقدام عامل می‌ماند.
- غیرقطعی (Deterministic): غیر قطعی است زیرا عامل شانس در آن وجود دارد.
- مشاهده‌پذیر (Fully Observable): عامل می‌تواند تمام اطلاعات مورد نیاز از وضعیت بازی را مشاهده کند و هیچ بخش پنهانی از محیط برای او وجود ندارد.
- گسسته (Discrete): وضعیت‌ها، اعمال و زمان در قالب مجموعه‌ای از حالات و حرکات مشخص و شمارا تعریف می‌شوند.
- غیر اپیزودیک یا ترتیبی (Sequential) است: تصمیمات هر مرحله بر وضعیت‌های بعدی بازی اثر می‌گذارند، بنابراین مسئله اپیزودیک (Episodic) محسوب نمی‌شود.

(۲)

با توجه به توضیحات ارائه شده در سوال اول، پاسخ این سوال به این شکل است:

- الف) از آنجا که اهداف متنوعی داریم، عامل از نوع عامل مبتنی بر سودمندی مناسب است.
- ب) با توجه به این که برای این اکشن نیازی به حافظه نداریم، پس ایجنت از نوع عامل واکنشی ساده می‌تواند مورد استفاده باشد.

در ابتدا سیاست های حل این سوال را بررسی می کنیم:

جستجوی درختی در برابر جستجوی گراف

از آنجا که در صورت سؤال به صراحت ذکر نشده جستجو باید به صورت graph search انجام شود، حالت پیش فرض یعنی جستجوی درختی (tree search) در نظر گرفته می شود. در این روش هیچ مجموعه ای از گره های ازپیش بازدید شده (ex-plored/closed set) نگه داری نمی شود؛ بنابراین ممکن است یک استتیت بیش از یک بار و با هزینه های g متفاوت در درخت ظاهر و حتی بسط داده شود، بدون آنکه نسخه های گران تر آن حذف گردند.

زمان انجام آزمون هدف

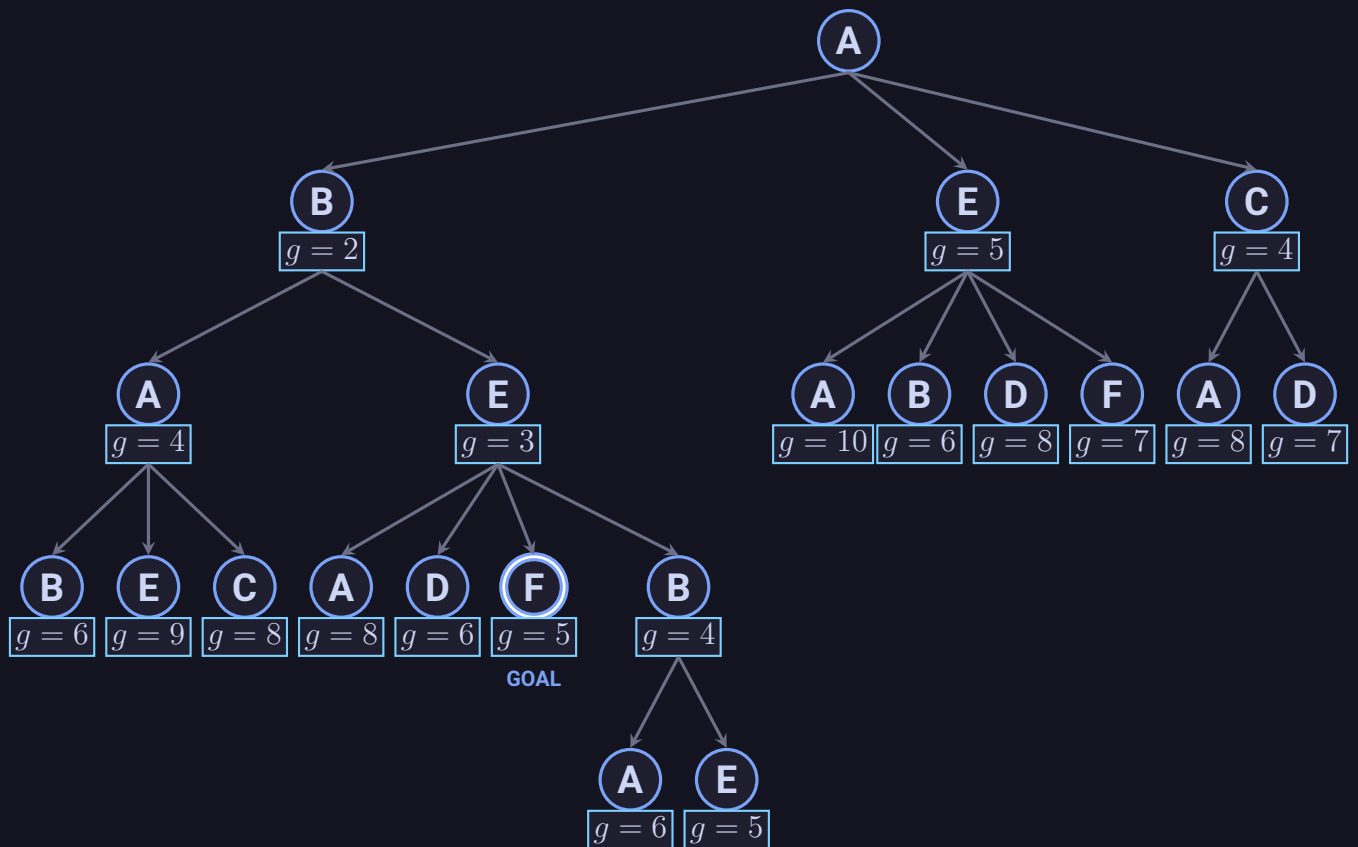
آزمون هدف هنگام بازدید (بیرون کشیدن از صف اولویت) انجام می شود، نه هنگام تولید گره. به بیان دیگر، اولین باری که گره F تولید می شود (به عنوان فرزند E با $g = 3$)، صرفاً وارد صف اولویت می گردد و الگوریتم بلافاصله متوقف نمی شود؛ تنها زمانی که نوبت به بیرون کشیدن آن به عنوان کمینه صف برسد و رسماً «بازدید» شود، پایان جستجو اعلام می گردد.

سیاست انتخاب در هزینه های برابر

هرگاه در صف اولویت دو یا چند گره با g برابر وجود داشته باشند، گره ای که زودتر تولید (زودتر وارد صف) شده است، زودتر بازدید می شود (سیاست FIFO میان گره های هم هزینه). البته توجه داشته باشید که این سیاست قابل تعیین است و می تواند بسته به مسئله تعیین شود و در صورتی که صراحتاً اعلام شود، می توان از سیاست دیگری هم استفاده کرد. (مثلاً رندوم و یا مثلاً بر اساس حروف الفبای گره ها)

تضمین بهینگی

به این سوال فکر کنید: آیا جستجوی UCS تضمین میکند که در صورتی که هزینه یالها نامنفی باشند، در حالت جستجوی گرافی بینه است؟ پاسخ: از آنجا که تمام وزن یال های گراف نامنفی هستند، طبق ویژگی UCS، اولین باری که هر حالت - چه در جستجوی گراف و چه در جستجوی درختی - از صف خارج و بازدید می شود، هزینه رسیدن به آن بهینه است. بازدیدهای بعدی همان حالت با هزینه بیشتر، صرفاً به دلیل ساختار درختی رخ می دهند و تأثیری در بهینگی پاسخ نهایی ندارند. با این حال این سوال به شکل درختی حل شده است و سوال بعدی به شکل گرافی حل شده تا تفاوت این دو نوع جستجو را بهتر متوجه شوید.



Step	Node Visited	Cost (g)	Nodes Generated
1	A	0	B(2), C(4), E(5)
2	B	2	A(4), E(3)
3	E	3	A(8), B(4), D(6), F(5)
4	C	4	A(8), D(7)
5	A	4	B(6), C(8), E(9)
6	B	4	A(6), E(5)
7	E	5	A(10), B(6), D(8), F(7)
8	F	5	– (Goal, stop)

• ترتیب بازدید گره‌ها:

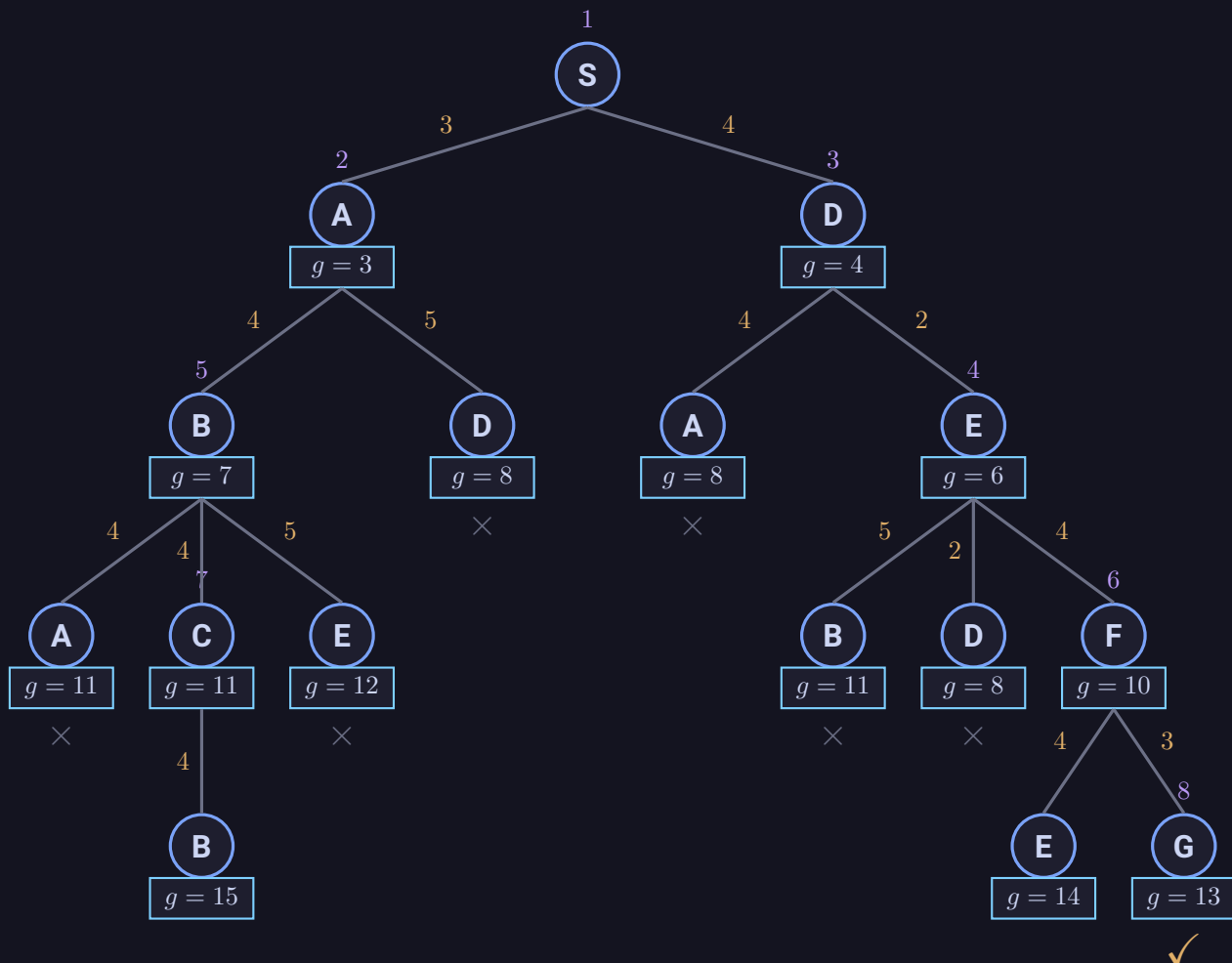
$A \rightarrow B \rightarrow E \rightarrow C \rightarrow A \rightarrow B \rightarrow E \rightarrow F$

که چون جستجو درختی انجام شده، برخی گره‌ها بیش از یکبار مشاهده شده است.

• مسیر بهینه:

$A \rightarrow B \rightarrow E \rightarrow F$

درخت جستجوی UCS مربوطه به صورت زیر است. توجه شود در جستجوی گرافی، گره ها به ترتیب هزینه مسیرشان انتخاب می شوند و گره هایی که قبلا بسط داده شده اند دیگر بررسی نمی شوند. این گره ها با علامت ضربدر نشان داده شده اند. اعداد بالای گره ها ترتیب بسط آنها را نشان می دهد.



با توجه به درخت ارائه شده، ترتیب پیمایش گره ها به ترتیب برابر خواهد بود با

S, A, D, E, B, F, C, G.

(۵)

می دانیم که روش هایی که با اندازه یال ها در گراف مسئله، به شکل یکنواختی برخورد می کنند، مناسب حل مسائل با یال های وزندار نیستند؛ پس روش های BFS و DFS و سایر روش هایی که از آنها مشتق می شوند مناسب حل این مسئله نخواهند بود. از طرفی، برای حل مسائلی با یال های وزن دار در گراف جست و جو، روش UCS روش کارایی است؛ اما از آنجا که در این مثال خاص، وزن ها منفی اند، به سادگی ممکن است در حلقه بی نهایت قرار بگیریم. بنابراین این مسئله با هیچ روش جستجوی کلاسیک قبل حل نیست.

(۶)

- **BFS** نمی تواند باشد، زیرا در Breadth-First Search گره ها بر اساس عمق گسترش داده می شوند؛ یعنی ابتدا تمام گره های عمق ۱، سپس عمق ۲ و بعد عمق ۳ بررسی می شوند. در شکل مشاهده می شود که یک گره در عمق بیشتر قبل از برخی گره های کم عمق تر گسترش یافته است، بنابراین این ترتیب با BFS سازگار نیست.
- **DFS** نیز نمی تواند باشد، زیرا در Depth-First Search الگوریتم ابتدا یک شاخه را تا انتها دنبال می کند و سپس به عقب بازمی گردد. در حالی که در شکل قبل از کامل شدن یک شاخه، شاخه های دیگر نیز گسترش یافته اند.
- **Iterative Deepening Search** هم نمی تواند باشد، زیرا این روش در هر مرحله تمام گره های یک عمق را بررسی می کند و سپس عمق جست و جو را افزایش می دهد؛ در نتیجه الگوی گسترش آن از نظر ترتیب عمق مشابه BFS است. اما در شکل چنین ترتیبی مشاهده نمی شود.
- **Uniform Cost Search (UCS)** می تواند این درخت را تولید کرده باشد، زیرا در این روش گره ها بر اساس کمترین هزینه مسیر (path cost) گسترش داده می شوند نه عمق. بنابراین ممکن است گرهی در عمق بیشتر ولی با هزینه کمتر زودتر از گره های کم عمق تر گسترش پیدا کند، که با الگوی مشاهده شده در شکل سازگار است.